

Gradient Descent:

Hill Analogy:



- standing on a foggy hillside
- feel the slope under your feet
- take a small step
- repeat until you reach the valley

In DL: loss is the hill surface;
weight is the position

The update rule:

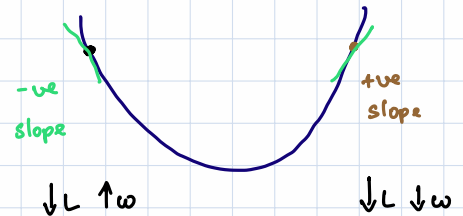
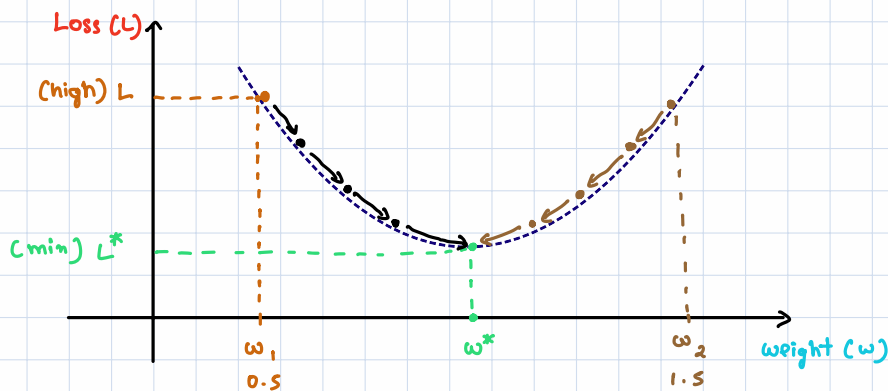
$$w \leftarrow w - \alpha \cdot \nabla L$$

w → weight we're adjusting

α → learning rate

∇L → gradient: points uphill [$\uparrow L$]

Gradient Descent - Visualization:



L^* → global minimum (lowest loss possible) (convergence)

w^* → weight that gives you the lowest loss

Note: This happens for all params ($w_1, w_2 \dots w_n$) together

The Training Flow:

1. Initialize weights randomly (eg: $w_1 = 0.5$)
 2. Forward Pass $\rightarrow$ compute Loss L
 3. Backward Pass $\rightarrow$ compute gradient ∇L
 4. Update : $w \leftarrow w - \alpha \cdot \nabla L$
 5. Repeat until loss is not reducing further (Convergence)
-



Common Pitfalls:

- $\rightarrow$ Local minima $\rightarrow$ stuck in a valley that's not the deepest
 - $\rightarrow$ Saddle Points $\rightarrow$ flat regions, gradient ≈ 0 ; loss looks minimum, but isn't
 - $\rightarrow$ Exploding gradients $\rightarrow$ α too large $\rightarrow$ loss shoots up
 - $\rightarrow$ Vanishing gradients $\rightarrow$ gradient $\rightarrow 0$ in deep layers
 $\rightarrow$ no learning happens
-